Assignment 6: Merge Commits and Conflicts

Overview

For this assignment, you will be continuing to practice branching and merging just like on the last assignment, with the addition of handling merge commits and merge conflicts that will come up. Learning how to resolve merge conflicts is very important, as you will experience them every so often throughout your programming life, and the key is to focus and work on a resolution instead of panicking that the code is "broken".

Assignment Instructions

Each student is to work on this assignment individually, however there are parts where you will need work with an assigned partner to make some commits to your repo.

Please read the assignment in full and look at the Submission requirements before starting to ensure that you know processes to screenshot as you work through the assignment. Do not forget to pull down **master** from the remote branch to your local **master** branch after a pull request is merged in – it is a critical piece of the branching flow!

Part 1: Repo Setup

Create a new repository for this assignment called **DmvSimulator**. Start the repo off with a blank Java file named **DmvSimulator.java**, but leave it empty (you can optionally start it off further by adding in a main method, etc.). You can push this initial commit directly to the **master** branch.

Create a branch off of **master** named **setup-simulator**. Inside the **DmvSimulator.java** class, create a program that:

- Welcomes the user to the DMV
- Gives the user a random number between 1 and 100 and tells them to wait until their number is called
- Call out each number one at a time, but be sure to start from the number directly after the one you gave to the user and then wrap around until the user's number is finally called last
- Tell the user they don't have the required paperwork

Add, commit, push, pull request, and merge the **setup-simulator** branch into the **master** branch.

Do not forget to switch back to the **master** branch and run a **git pull** in your local repo before moving on!

Part 2: Creating a Merge Commit

Create another branch called **increase-wait-time**. Make a change that increases the range of the random number you give to the user from 100 to 200 (much more accurate). Add, commit, and push this change to the branch, but **do not create a pull request or merge it in yet**.

Add your assigned partner to the repo as a collaborator, and have them clone your repo. Your partner should create a branch called **final-message-change** and change the final message (the one that tells the user they don't have the required paper work) to be a lot meaner and nastier, as we are aiming for a realistic simulation here. They should then test run the program to ensure it works, push this change up, create a pull request, and then merge the change into the **master** branch.

After your partner has merged their change to **master**, it is time to merge in your **increase-wait-time** branch into **master**. First, run a **git pull origin master** command to bring down any new changes from the **master** branch into your **increase-wait-time** branch to keep it in sync prior to creating a pull request (you should ALWAYS do this before creating a pull request – get in the habit!!!). You should see a message saying "Auto-merging **DmvSimulator.java**", which is what creates the merge commit. It'll look like the below example:

```
E:\Documents\dmvb\DmvSimulator (increase-wait-time)
λ git pull origin master
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), 698 bytes | 4.00 KiB/s, done.
From https://github.com/CSC109/DmvSimulator
 * branch            master     -> FETCH_HEAD
   5f4b4db..31a8ee2  master     -> origin/master
Auto-merging DmvSimulator.java
Merge made by the 'recursive' strategy.
 DmvSimulator.java | 4 ++--
 1 file changed, 2 insertions(+), 2 deletions(-)
```

Note: you may see Git display an error when attempting to run **git pull origin master** that looks like the below image:

```
hint: You have divergent branches and need to specify how to reconcile them.
hint: You can do so by running one of the following commands sometime before
hint: your next pull:
hint:
hint:   git config pull.rebase false  # merge
hint:   git config pull.rebase true   # rebase
hint:   git config pull.ff only       # fast-forward only
hint:
hint: You can replace "git config" with "git config --global" to set a default
hint: preference for all repositories. You can also pass --rebase, --no-rebase,
hint: or --ff-only on the command line to override the configured default per
hint: invocation.
fatal: Need to specify how to reconcile divergent branches.
```

If you see this, it means you need to tell **git** how to handle merges, since it supports multiple different "merging strategies". I recommend that you run the command **git config pull.rebase false**, which will stick to the "default" strategy of creating merge commits (which is what I taught you in class 😊).

```
E:\Documents\dmvb\DmvSimulator (master -> origin)
λ git config pull.rebase false
```

This will stop that error from popping up from now on in this repo. Run the **git pull origin master** command again and it will work.

While you are at it, I recommend that you run the command **git config –global pull.rebase false** to make it so you will never run into this error again for any future repos:

```
C:\Users\alext\Documents\dmvb\DmvSimulator2 (master -> origin)
λ git config --global pull.rebase false
```

You can confirm the merge commit was created by running a **git log -1**. Below is an example of what you should see:

```
E:\Documents\dmvb\DmvSimulator (increase-wait-time)
λ git log -1
commit b8e484ed2df013188c0e3af1679c183a8aee783e (HEAD -> increase-wait-time)
Merge: c2510e8 31a8ee2
Author: a-r-t <alexthimineur@gmail.com>
Date:   Mon Feb 27 20:25:14 2023 -0500

    Merge branch 'master' of https://github.com/CSC109/DmvSimulator into increase-wait-time
```

The command **git log -1** shows the most recent commit

The merge commit had to be created since your peer added a change to the **master** branch that your **increase-wait-time** branch did not know about until it ran the **git pull origin master** command to sync up with it. However, since their change didn't occur on the same line as something you changed, Git will handle this merge for you (there will be no merge conflict).

The important thing to do from here after synching up your branch with the **master** branch and creating a merge commit is to re-test your current changes and ensure everything still works as you. If everything still looks good, run one final **git push** to push the merge commit back up to GitHub, and then create your pull request and merge in your **increase-wait-time** branch into **master**. Woo, you just worked together with someone else asynchronously!!!

Do not forget to switch back to the **master** branch and run a **git pull** in your local repo before moving on!

Part 3: Creating and Resolving a Merge Conflict

Create *another* branch called **add-miracle** that changes the end of the program to have a 99% chance of telling the user the original message (forgotten paperwork – do not change the wording of what's currently there), and a 1% chance of telling the user that they have all of the required paper work and that they are all set. Add, commit, and push this change, but **do not create a pull request or merge it in yet**.

Have your assigned partner make another branch called **evil-laugh** and have them change the forgotten paperwork message to include an evil laugh at the end. They should then push this change up, create a pull request, and then merge the change into the **master** branch.

Once again, after the peer has merged their change to **master**, you are now going to merge in your changes. First, run a **git pull origin master** command to bring down changes from the **master** branch that your **add-miracle** branch doesn't currently have, as it's important to sync up your branch before creating the pull request. And here is where the merge conflict comes in (assuming you followed directions). Your **git pull** command should yield an output that looks like this, where it says:

Auto-merging **DmvSimulator.java**
CONFLICT (content): Merge conflict in **DmvSimulator.java**
Automatic merge failed; fix conflicts and then commit the result.

```
E:\Documents\dmvb\DmvSimulator (add-miracle)
λ git pull origin master
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), 657 bytes | 4.00 KiB/s, done.
From https://github.com/CSC109/DmvSimulator
 * branch            master     -> FETCH_HEAD
   9df45ea..b2d484a  master     -> origin/master
Auto-merging DmvSimulator.java
CONFLICT (content): Merge conflict in DmvSimulator.java
Automatic merge failed; fix conflicts and then commit the result.
```

Since your partner made a change on the same line of the same file that you did, and their change was merged into the **master** branch after you created your commit, Git cannot merge this together as it does not want to cause a "loss of work". It is now up to you to manually fix this merge conflict in the **DmvSimulator.java** file.

It is important that you do NOT remove your partner's changes – the goal is to merge your partner's changes and your changes together.

After resolving the merge conflict, test the program to ensure it works correctly. If all looks good, create a commit with the message **"resolving merge conflicts"**, push this commit up to GitHub, and then carry out the usual steps to merge your **add-miracle** branch into **master**.

Submission

Your final submission to Blackboard should include:

- A link to your GitHub repository's webpage
- Screenshot of your git pull command causing a merge conflict in the **add-miracle** branch.
- Screenshot of the merge conflict in your **DmvSimulator.java** file before it is resolved.
- Screenshot of the merge conflict in your **DmvSimulator.java** file after it is resolved.
- Links to your two pull requests in your partner's repo.

Grading Rubric

In order to get an A on this assignment (20/20):

- Merge commit was created successfully: **7pts**
- Merge conflict was created and resolved successfully (screenshots included): **7pts**
- All assignment directions and sequences were followed: **6pts**

Note: If you do not work with your partner to make the needed commits in their repo, you will receive a zero on this assignment.